DocumentDB Prod CPU Incident — Root Cause & Resolution

Nov 11, 2026 · @Juhi Sinha

Summary

Missing indexes on two DocumentDB collections caused full collection scans that pinned `docdb-production-ttn` CPU near 100% for over an hour starting around 08:50 IST. The scans were identified via Performance Insights, confirmed via `explain`, and fixed by building two indexes (`investorRequestId_1` on `prod_submit_payload`, `userActivityId_1` on `prod_user_activity_payload_detail`). Both are now verified with `IXSCAN` and sub-10ms execution. The database-side issue is resolved. Three application-side issues contributed to the severity and remain open: no query timeout, a retry/polling loop re-issuing the same query for over an hour, and a type mismatch causing silent empty results.

Timeline

Time	Event
~08:50	`docdb-production-ttn` CPU jumps from ~20% to ~100% and stays there
09:30	CloudWatch graph and Performance Insights confirm sustained CPU saturation on the primary
~10:30	Performance Insights shows `find` on `prod_submit_payload` filtered by `investorRequestId` as top load by wait (AAS); 9 copies of this query found running 30-82 minutes each via `currentOp`
~10:48	9 long-running scans killed with `killOp`; CPU drops from 100% to ~65%
~10:50	`createIndex({investorRequestId: 1})` started on `prod_submit_payload`

Time	Event
~13:33	First index build (`investorRequestId_1`) completes; `explain` confirms `IXSCAN` , 5.9ms
~15:43	A second runaway scan found on `prod_user_activity_payload_detail` (`userActivityId`), running 65 minutes; killed
~15:43	`createIndex({userActivityId: 1})` started on `prod_user_activity_payload_detail` (4.16M-block collection)
~19:40	Second index build (`userActivityId_1`) completes; `explain` confirms `IXSCAN` , 5.2ms

Root cause

Primary cause — missing indexes. Two collections had queries filtering on a field with no supporting index:

Collection	Query field	Index before	Result
`prod_submit_payload`	`investorRequestId`	`_id_` only	Full `COLLSCAN` per lookup
`prod_user_activity_payload_detail`	`userActivityId`	`_id_` , `requestDetailId`	Full `COLLSCAN` per lookup

Every lookup on these fields scanned the entire collection. Under normal traffic this was slow but tolerable; once query volume or collection size crossed a threshold, scans began queuing and the primary's CPU saturated.

Contributing causes — application behavior. These did not cause the scans but turned a slow query into an hours-long outage:

- **No query timeout.** Individual scans were observed running 16 minutes, 65 minutes, and up to 82 minutes before being manually killed. A `maxTimeMS` setting would have capped each at a few seconds.
- **Retry/polling without backoff.** The same query (same `lsid` , same filter value) was re-issued repeatedly over more than an hour, multiplying load from a single logical request into 5+ concurrent scans.

- **Type mismatch.** The application queries `investorRequestId` as a string (e.g. `"3561010"`), but the field is stored as `NumberLong`. A string filter against a `NumberLong` field returns zero results even when matching documents exist — a silent correctness bug that may also explain some of the retry behavior (a caller treating an empty result as "not found yet" and retrying).

Diagnosis steps

1. **CloudWatch CPUUtilization graph** for `DocumentDB-Prod` showed `docdb-production-ttn` flat at ~15-20% then jumping to ~100% at 08:50 and holding, while the sibling instance stayed low — isolating the load to one primary node.
2. **Performance Insights "Load by wait (AAS)"** identified the top query: `find` on `prod_submit_payload` filtered by `investorRequestId`, accounting for ~4+ AAS, with 5+ concurrent copies of the exact same filter value.
3. **`getIndexes()`** on `prod_submit_payload` showed only the default `_id_` index — confirming no index existed on `investorRequestId`.
4. **`explain("executionStats")`** on the query (correctly scoped to `prod_sws_db`, not the default `admin` database) would have shown `COLLSCAN`, but the query itself ran long enough that `currentOp` was used instead to inspect it live.
5. **`currentOp({"ns": ..., "op": "query", "secs_running": {$gt: 5}})`** surfaced 9 queries that had been running 30-82 minutes each — these were killed with `killOp` to relieve CPU before building the index.
6. The same pattern was found a second time via Performance Insights on `prod_user_activity_payload_detail` (`userActivityId`), confirmed the same way: `getIndexes()` showed no supporting index, `currentOp` found a 65-minute scan, which was killed.

Resolution

Two indexes were created and verified:

Collection	Index	Build time	Before	After
`prod_submit_payload`	`investorRequestId_1`	~2h50m (580 blocks)	`COLLSCAN`, queries running 30-82 min	`IXSCAN`, 5.875ms
`prod_user_activity_payload_detail`	`userActivityId_1`	~5h30m (4.16M blocks)	`COLLSCAN`, query running 65 min	`IXSCAN`, 5.167ms

```
// Index 1
var d = db.getSiblingDB("prod_sws_db")
d.prod_submit_payload.createIndex({ investorRequestId: 1 })

// Index 2
d.prod_user_activity_payload_detail.createIndex({ userActivityId: 1 })
```

Both builds were run with `createIndex` (foreground) rather than `background: true`, since DocumentDB does not require the legacy background option. Progress was tracked via `currentOp({"command.createIndexes": {$exists: true}})`, which reports scan percentage. Runaway scans that reappeared on the target collections during each build were identified and killed via `killOp` so they wouldn't compete with the build for CPU.

After both builds completed, CPU on `docdb-production-ttn` returned to its pre-incident baseline (~15-20%).

Open action items

Item	Owner	Priority	Notes
Add `maxTimeMS` (~5000ms) and a socket timeout to queries on `prod_submit_payload` and `prod_user_activity_payload_detail`	App team	High	Would have capped every scan today at a few seconds instead of up to 82 minutes
Add retry backoff / dedupe repeated lookups for the same ID	App team	High	Same `investorRequestId` / `userActivityId` was re-queried identically, same session, for over an hour

Item	Owner	Priority	Notes
Fix type mismatch: send `investorRequestId` as `NumberLong`, not a string	App team	High	Current string queries return 0 results silently even when matching documents exist
Identify what changed around 08:50 (deployment, batch job, traffic burst) that triggered the spike	App/Infra	Medium	Root trigger not yet identified
Add CloudWatch alarm on CPUUtilization > 80% for 10+ minutes	Infra/DBA	Medium	No early warning before CPU saturated for over an hour
Audit other collections hit by `prod_sws_db` traffic for missing indexes	DBA	Medium	Only two collections were checked reactively; others may have the same gap
Consider instance sizing review for `docdb-production-ttn` given current data volume (4M+ documents in one collection)	Infra	Low	CPU swung into 85-90% even during index builds alone

Reference commands

```
// Always scope to the correct database first
var d = db.getSiblingDB("prod_sws_db")

// Check indexes on a collection
d.<collection>.getIndexes()

// Check query plan without executing (safe, no scan)
d.<collection>.find({ field: value }).explain()

// Check query plan WITH execution stats (executes the query)
d.<collection>.find({ field: value }).explain("executionStats")

// Count currently running slow queries on a collection
d.currentOp({ "ns": "prod_sws_db.<collection>", "op": "query",
"secs_running": { $gt: 5 } }).inprog.length

// Inspect slow queries in detail (across all collections)
d.currentOp({ "op": "query", "secs_running": { $gt: 5 }
}).inprog.forEach(function(o) {
  printjson({ opid: o.opid, ns: o.ns, secs: o.secs_running, cmd: o.command })
})
```

```
// Kill a specific runaway operation (never kill an index build this way)
d.killOp(<opid>)

// Kill all slow queries on one collection at once
d.currentOp({ "ns": "prod_sws_db.<collection>", "op": "query",
"secs_running": { $gt: 5 } })
  .inprog.forEach(function(o) { print(o.opid + "  " + o.secs_running + "s");
d.killOp(o.opid) })

// Create an index
d.<collection>.createIndex({ field: 1 })

// Track an index build's progress
d.currentOp({ "command.createIndexes": { $exists: true } })
  .inprog.forEach(function(o) { printjson({ opid: o.opid, secs:
o.secs_running, msg: o.msg }) })
```

Rules of thumb used throughout this incident:

- Never kill the `createIndexes` operation's opid — an unfinished build just rolls back, costing time, not data.
- Never run `createIndex` twice in parallel for the same or different indexes — concurrent builds compete for CPU and slow each other down.
- Confirm the `ns` and `cmd` of a slow op before killing it — don't kill blind.
- A query's filter value's data type must match the stored field's type exactly (e.g. `NumberLong(123)` vs `"123"`) or it silently returns zero matches, index or not.